CT-159 · DATA STRUCTURES

Arrays, Linear Search & Binary Search

Lecture 1 (Part B) — Arrays & Searching Algorithms

Course Teacher: Dr. Muhammad Kamran

SECTION 01

Arrays

A collection of same-type values stored in one contiguous block of memory.

SECTION 01 · ARRAYS

01

What is an Array?

An array is a collection of variables of the same type, referred to by a common name.

Derived type	Built from primitive types — a derived data structure.
Contiguous	Elements sit in consecutive memory locations.
Homogeneous	Every element is of the same data type.
Zero-indexed	Indexed from 0 up to (size − 1).

SECTION 01 · ARRAYS

02

Contiguous vs Non-Contiguous

Contiguous	Non-Contiguous
• Consecutive blocks for a file/process	• Separate scattered blocks
• Faster in execution	• Slower in execution
• Easier for the OS to control	• Harder for the OS to control
• Some memory wastage possible	• Little to no memory wastage

SECTION 01 · ARRAYS

03

How an Array Sits in Memory

Elements are laid out consecutively — the index maps directly to a memory address.

int num[7] — one contiguous block

11	22	33	44	55	66	77
0	1	2	3	4	5	6
2000	2004	2008	2012	2016	2020	2024

index (subscript)

memory address (4 bytes per int)

num[3]

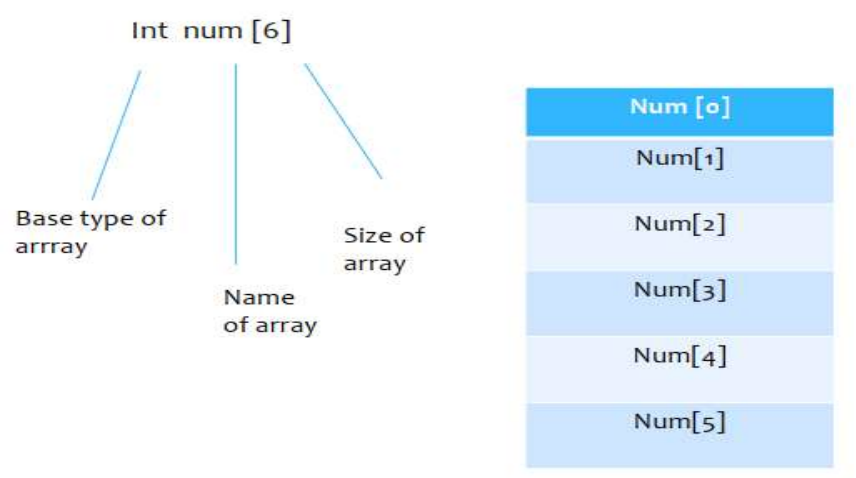
address = base(2000) + 3 × 4 = 2012 → O(1) access

SECTION 01 · ARRAYS

04

How an Array Sits in Memory

Elements are laid out consecutively — the index maps directly to a memory address.



SECTION 01 · ARRAYS

05

Why Do We Need Arrays?

- Store a large number of same-type variables under one name.
- Make programs easier to read and reason about.
- Provide O(1) random access — reaching index 0 and index 1000 take the same time.

Access is instant because address = base_address + index × element_size.

SECTION 01 · ARRAYS

06

Types of Array

One-Dimensional	A single row of elements — one subscript.
Two-Dimensional	A grid of rows and columns — two subscripts.
Multi-Dimensional	Three or more subscripts — rarely used (memory-heavy).

SECTION 01 · ARRAYS

07

One-Dimensional Array

- One subscript / index specifies a particular element of the array.

```
Data_type  name[size];
```

Examples

```
int  num[20];
char name[25];
```

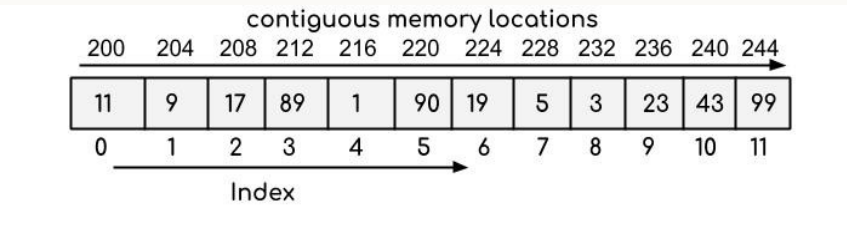
SECTION 01 · ARRAYS

08

Memory Size of a 1D Array

size of array = length × size of (base type)

e.g. int num[20] on a 4-byte-int system → 20 × 4 = 80 bytes.



SECTION 01 · ARRAYS

09

Two-Dimensional Array

- A 2D array is an array in which each element is itself an array (a grid of rows × columns).

```
Data_type  name[row][col];
```

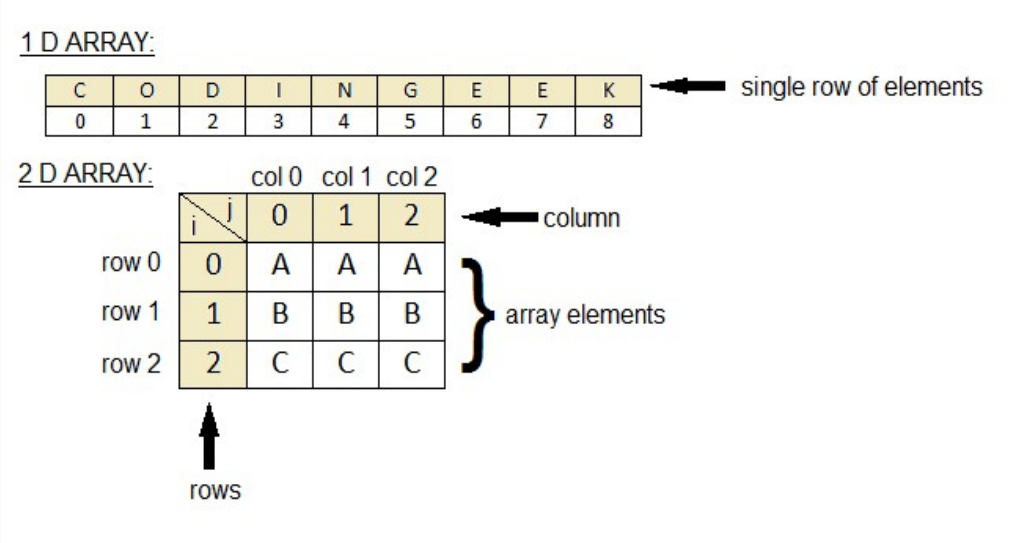
Example

```
int num[2][4];
```

SECTION 01 · ARRAYS

10

1D vs 2D Arrays



SECTION 01 · ARRAYS

11

Row-Major vs Column-Major

Row-major	Successive elements go across each row, then down to the next row (row-wise).
Column-major	Successive elements go down each column, then across to the next column.

A 2D array is logically a grid, but memory is linear — the ordering decides the layout.

SECTION 01 · ARRAYS

12

Row-Major vs Column-Major — Layout

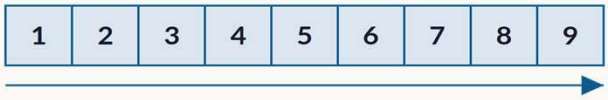
The same 3×3 matrix, laid out two different ways in linear memory.

Logical 2D array

1	2	3
4	5	6
7	8	9

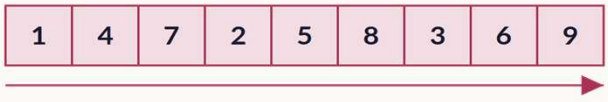
Row-major order

across each row, then down to the next row



Column-major order

down each column, then across to the next column

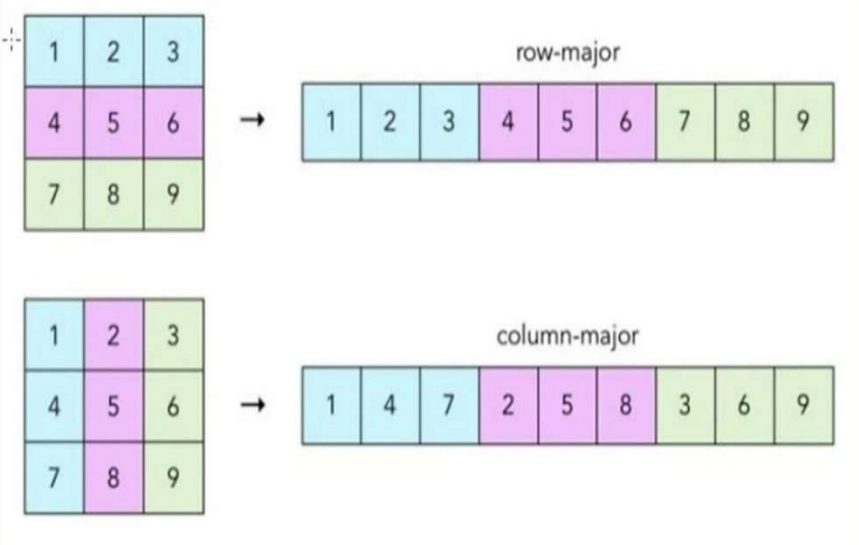


Both store the same matrix — they differ only in the order elements are laid out in linear memory.

SECTION 01 · ARRAYS

13

Row-Major vs Column-Major — Layout

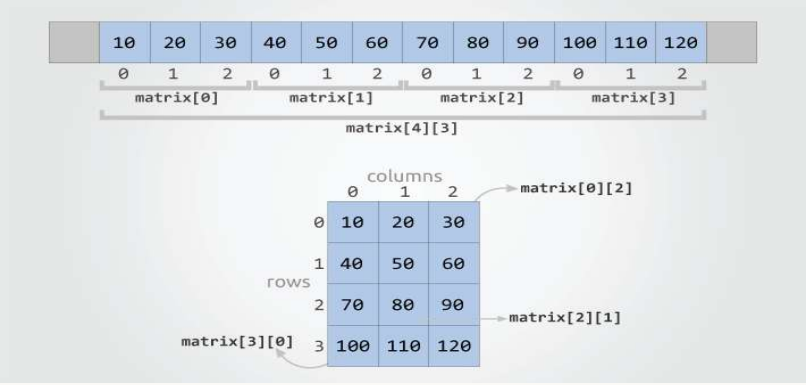


SECTION 01 · ARRAYS

14

Memory Size of a 2D Array

total bytes = rows × columns × size of (base type)

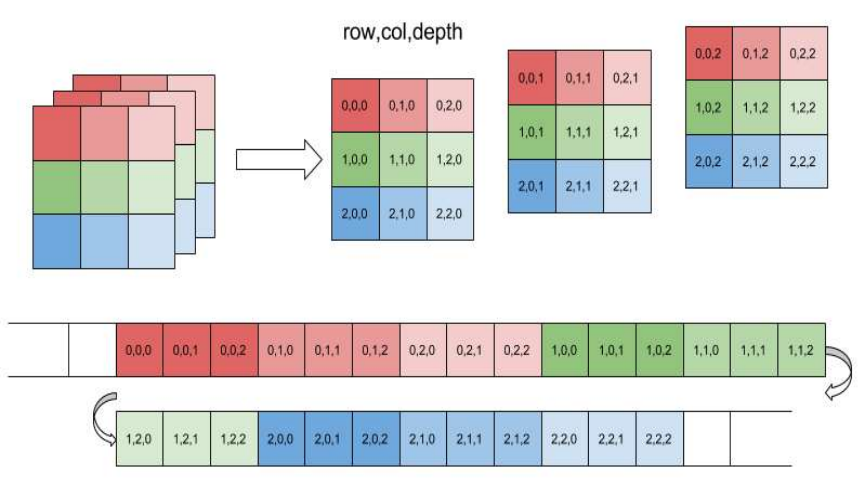


e.g. int num[2][4] on a 4-byte-int system → 2 × 4 × 4 = 32 bytes.

SECTION 01 · ARRAYS

Multi-Dimensional Array

- An array with more than two dimensions; the maximum is compiler-dependent.
- Declared as array_name[a][b][c][d]…[n].
- Arrays of 3+ dimensions are rarely used — high memory cost and complexity.



SECTION 02

Searching

Finding an item in a collection — from scanning to halving.

SECTION 02 · SEARCHING

16

A Familiar Problem

How do you find a phone number in a phone book?

Searching for an item in a collection of data is one of the most common tasks in computing — the same idea as looking up a record in a database.

SECTION 02 · SEARCHING

17

Approach 1 — Check Every Page

```
findNumber(person) {
    for (p = first page; p <= last page; p++) {
        if (person is found on page p) {
            return the person's phone number
        }
    }
    return NOT_FOUND
}
```

SECTION 02 · SEARCHING

18

Approach 2 — Split the Book in Half

```
findNumber(person) {
    min = first page
    max = last page
    while (min <= max) {
        mid = (min + max) / 2          // middle page
        if (person is found on page mid) {
            return the person's number
        } else if (name comes earlier) {
            max = mid - 1              // search left half
        } else {
            min = mid + 1              // search right half
        }
    }
    return NOT_FOUND
}
```

SECTION 02 · SEARCHING

19

Two Named Approaches

Approach 1 → Linear Search	Check items one by one. Also called sequential search. Works on any data.
Approach 2 → Binary Search	Repeatedly halve the search space. Requires the data to be sorted first.

SECTION 02 · SEARCHING

20

Categories of Search

Sequential Search	The list is traversed in order; every element is checked. Simple, works unsorted.
Interval Search	Designed for sorted data. Repeatedly targets the centre and halves the space — far more efficient.

SECTION 02 · SEARCHING

21

Linear Search

Scan the collection from start to end, checking each element until a match is found.

- The simplest search algorithm — a sequential pass over all items.
- Returns the item when matched; otherwise continues to the end of the collection.
- No ordering required — works on unsorted data.

SECTION 02 · SEARCHING

22

Linear Search — Pseudocode

```
Linear Search (Array A, Value x)

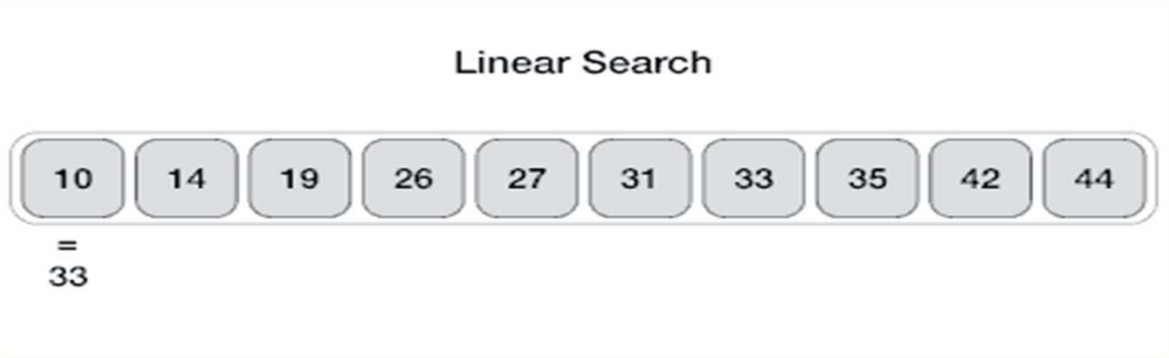
  Step 1:  Set i = 1
  Step 2:  if i > n, go to Step 7
  Step 3:  if A[i] = x, go to Step 6
  Step 4:  Set i = i + 1
  Step 5:  Go to Step 2
  Step 6:  Print "Found at index i", go to Step 8
  Step 7:  Print "Element not found"
  Step 8:  Exit
```

SECTION 02 · SEARCHING

23

Linear Search — Step by Step

The target is compared with each element in turn until it is found.

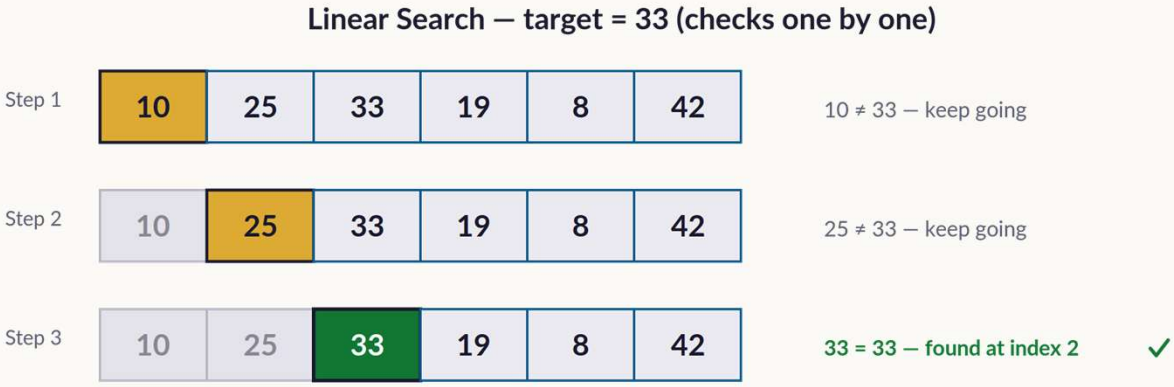


SECTION 02 · SEARCHING

24

Linear Search — Step by Step

The target is compared with each element in turn until it is found.



Worst case: the target is last (or absent) — every element is checked → O(n).

SECTION 02 · SEARCHING

25

Binary Search

Repeatedly halve a sorted array, comparing the target with the middle element.

- Finds the position of a target value within a sorted array.
- Each comparison discards half of the remaining elements.
- Continues until the value is found or the interval becomes empty.

SECTION 02 · SEARCHING

26

When Can You Use Binary Search?

Sorted data	The structure must already be sorted before searching.
Constant-time access	Reaching any element by index must take O(1) time (e.g. an array).

SECTION 02 · SEARCHING

27

Binary Search — The Idea

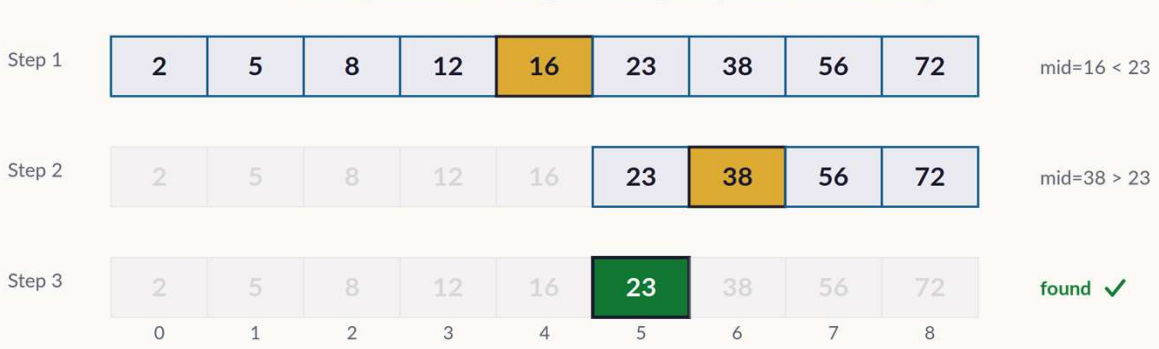
- Divide the search space into two halves by finding the middle index “mid”.
- Compare the middle element of the search space with the key.
- If the key is found at the middle element, the process is terminated.
- If the key is not found at the middle element, choose which half will be used as the next search space.
  - If the key is smaller than the middle element, then the left side is used for the next search.
  - If the key is larger than the middle element, then the right side is used for the next search.
- This process is continued until the key is found or the total search space is exhausted.

SECTION 02 · SEARCHING

28

Binary Search — Step by Step

```
mid = low + (high - low) / 2
```



Each step halves the remaining search space → O(log n)

9 elements found in 3 steps instead of up to 9 — the advantage grows fast with size.

SECTION 02 · SEARCHING

29

Iterative Binary Search (C++)

```
int binarySearch(int arr[], int low, int high, int x) {
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x)                  // found
            return mid;
        if (arr[mid] < x)                   // ignore left half
            low = mid + 1;
        else                                // ignore right half
            high = mid - 1;
    }
    return -1;                              // not present
}
```

SECTION 02 · SEARCHING

30

Recursive Binary Search (C++)

```
int binarySearch(int arr[], int low, int high, int x) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x)                  // found
            return mid;
        if (arr[mid] > x)                   // search left
            return binarySearch(arr, low, mid - 1, x);
        return binarySearch(arr, mid + 1, high, x); // right
    }
    return -1;                              // not present
}
```

In recursive approach, we create a recursive function and compare the mid of the search space with the key. And based on the result either return the index where the key is found or call the recursive function for the next search space.

SECTION 02 · SEARCHING

31

Why Binary Search Wins

- Simple to implement, yet a big improvement over linear search.
- Performs comparably to much more complex search algorithms.
- Halves the list every iteration instead of scanning it — the gain grows fast on large lists.

SECTION 02 · SEARCHING

32

Time Complexity — Linear vs Binary

Linear Search	Binary Search
O(n)	**O(log n)**
Checks up to every element. Doubling the data doubles the work.	Halves the data each step. Doubling the data adds just one step.

Requires sorted data

SECTION 02 · SEARCHING

33

Time Complexity Derivation — Binary

For each recurrence in the recurrence relation for binary search, we convert the problem into one subproblem, with runtime T(N/2). Therefore:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Substituting into the master theorem, we get:

$$\mathrm{T(n)} = \mathrm{aT}\left(\frac{n}{b}\right) + f(n)$$
$$a = 1, b = 2, f(n) = 1$$

Now, because $log_b a$ is 0 and f(n) is 1, we can use the second case of the master theorem because:

$$f(n) = O(1) = O(n^0) = O(n^{log_b a})$$

This means that:

$$T(n) = O(n^{log_b a} log(n)) = O(n^0 log(n)) = O(log(n))$$

SECTION 02 · SEARCHING

34

Master Theorem (for recurrences)

$$T(n) = a \cdot T(n/b) + f(n)$$

Case 1	$f(n) = O(n^{\log_b a - \varepsilon})$	→ $T(n) = \Theta(n^{\log_b a})$
Case 2	$f(n) = \Theta(n^{\log_b a})$	→ $T(n) = \Theta(n^{\log_b a} \cdot \log n)$
Case 3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$	→ $T(n) = \Theta(f(n))$

SECTION 03

Practice It Yourself

The only way to learn DSA is to solve problems by hand!

PRACTICE · SEARCHING

P1

Problems to Try

Solve these yourself first.

LeetCode 704	**Binary Search**	Easy	Find a target in a sorted array; return its index or -1.
LeetCode 35	**Search Insert Position**	Easy	Return the index where a target is, or where it would be inserted.
LeetCode 278	**First Bad Version**	Easy	Binary-search the first failing version using a given API.
LeetCode 74	**Search a 2D Matrix**	Medium	Treat a sorted matrix as one long array and binary-search it.
Codeforces 1409B	**Minimum Product**	Easy	Reason about halving and bounds — good binary-search warm-up.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 704 — The Approach

Problem (in short): given a sorted array nums and a target, return the index of target, or -1 if it isn't there.

- Because the array is sorted, we never need to check every element.
- Keep a window [low, high]. Look at the middle element mid.
- If nums[mid] == target → found it, return mid.
- If nums[mid] < target → the answer must be to the right → low = mid + 1.
- If nums[mid] > target → the answer must be to the left → high = mid - 1.
- Repeat until the window is empty (low > high) → return -1.

YOUR TURN

Before looking at the code: on paper, binary-search for 9 in [1,3,5,7,9,11]. Write down low, high and mid at each step. How many comparisons did it take?

PRACTICE · WORKED EXAMPLE

P3

LeetCode 704 — The Code

```
int search(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;   // avoids overflow

        if (nums[mid] == target)
            return mid;                     // found
        else if (nums[mid] < target)
            low = mid + 1;                  // go right
        else
            high = mid - 1;                 // go left
    }
    return -1;                              // not found
}
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

Correctness	Each step discards a half that cannot contain the target, so the window always keeps the answer if it exists.
mid formula	low + (high - low)/2 finds the midpoint without integer overflow on large indices.
Time	O(log n) — the window halves every step. 1,000,000 items take about 20 comparisons.
Space	O(1) — only a few integer variables, no extra memory.

PRACTICE · RESOURCES

P5

Where to Get Help

Use these when you're stuck after trying — watch the idea, then code it yourself.

NeetCode	Clear, visual walk-throughs. Search: "NeetCode Binary Search 704"
Abdul Bari	Deep algorithm intuition. Search: "Abdul Bari Binary Search"
William Fiset	Data-structures playlist for CS students. Search: "William Fiset Binary Search"

YOUR TURN

Rule for this course: try a problem for at least 20 minutes before watching any video — and never paste it into an AI. Struggle yourself to build your logical skills.

LECTURE 1B — RECAP

What we covered

- Arrays: contiguous memory & O(1) access
- 1D, 2D and multi-dimensional arrays
- Row-major vs column-major layout
- Linear search — O(n), works unsorted
- Binary search — O(log n), needs sorted data

Next: Linked Lists